

CTMM — Consolidated Claims

CHURCH-TURING META-MACHINE, CLOOMC Capability-Limited/Object-Oriented/Machine- Code

Consolidated Claims Document for the Patent Office

Inventor: Kenneth James Hamer-Hodges

Search: February – April 2026 (four applications, consolidated below)

Classification: Computer Architecture · Distributed Digital Security · Hardware & Software · Networked Security · Capability-Based Computing · Lambda Calculus Processor · Compiler Architecture · I/O Virtualization · Network Security · Vulnerability Elimination by Construction

SUMMARY OF THE INVENTION

The Church-Turing Meta-Machine (CTMM) uses CLOOMC, fail-safe, machine code in a processor architecture that enforces all access control through unforgeable capability tokens called **Golden Tokens (GTs)**, validated by a dual-gate Trusted Security Base. The architecture integrates Church's lambda calculus with Turing's computational model through clean domain separation that flawlessly eliminates entire classes of known vulnerabilities in computer science by proactive design construction rather than by reactive software mitigation.

Core Principles

1. Golden Token Architecture Every capability is an unforgeable 128-bit token with a 2-bit Type field (Inform, Outform, NULL, Abstract) and six permission bits organised into mutually exclusive Turing-domain (R Read, W Write, X Execute) and Church-domain (L Load, S Save, E Enter) sets. Capability registers hold capabilities exclusively; data registers hold values exclusively.

2. Dual-Gate Trusted Security Base The entire trusted computing base is two hardware gates, totalling fewer than 400 lines of synthesisable HDL — five orders of magnitude smaller than Linux, two orders of magnitude smaller than seL4 and adding the additional power of secure, trusted Lambda Calculus mathematics.

- **mLoad (Read Gate):** Validates every read-side capability operation: permission check → bounds check → version match → MAC/seal validation → G-bit reset → register write → thread shadow update.
- **mSave (Write Gate):** Validates every write of a GT to a C-List: source version match → seal validation → target bounds check → B-bit (bind) check → F-bit routing → seal recomputation → G-bit reset → commit, and B-bit authority to share and save a GT.

3. LAMBDA Instruction A dedicated hardware instruction implementing Church's function application within the current protection domain, using Execute (X) permission. Unlike CALL (which crosses domain boundaries with a full stack frame), LAMBDA saves only a return address to machine-status registers and branches to the code body at near-zero overhead. This makes Church's lambda calculus a first-class hardware primitive.

4. Domain Purity — Church meets Turing The architecture unifies both foundational computational models. Turing-domain instructions (DREAD, DWRITE, arithmetic, branching) operate on data. Church-domain instructions (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM) operate on capabilities. A GT cannot simultaneously hold Turing and Church permissions. Church is the armour (interface, security); Turing is the sword inside (implementation, hidden and atomic).

5. Pure Church Variant — Security by Exclusion In its Pure Church variant the processor exposes only six instructions to software (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM), architecturally excluding all arithmetic, branching, and memory-addressing instructions. Computational completeness is achieved through Church-encoded lambda calculus. This eliminates buffer overflows, return-oriented programming, code injection, and privilege escalation **by construction** — the instructions needed to mount the attacks do not exist.

6. Zero-OS Atomic Abstraction Architecture No central operating system, virtual memory, privilege rings, or superuser. All system services are atomic abstractions accessed exclusively through Golden Tokens. The architecture achieves seven security zeros: zero OS required, zero virtual memory, zero

privilege escalation, zero superuser, zero unauthorised code execution, zero unauthorised data access, zero containment escape.

7. Universal Computation Target A small 20-instruction set — 10 Church-domain, 10 Turing-domain — to which programs written in fundamentally different paradigms (imperative JavaScript, functional Haskell, natural-language English, symbolic machine code) compile through a multi-front-end compiler (CLOOMC++). The compiler is architecturally outside the trusted computing base: no compiler bug can produce a security violation.

8. Abstract GT I/O and Network Addressing A novel GT type (Abstract, `gt_type = 112`) whose location field holds a hardware-routed sentinel address rather than a namespace slot index. This unifies I/O peripherals, network tunnels, and system resources under the same unforgeable capability model. A single Home Base Tunnel (0xFF000000) is the sole outbound network gateway. Structural capability scoping — not policy filtering — enables verifiably crime-free services.

9. Idempotent LAMBDA Recursion LAMBDA CR6 (the capability list register established by the CALL lump split) provides optimal $O(1)$ recursive self-invocation: the return address written on each re-entry is invariant, making each re-entry idempotent. Two RETURN instructions suffice for any recursion depth. No stack frames, no hardware counter, no additional registers beyond those already present for single-level LAMBDA.

10. Deterministic Garbage Collection A four-phase (Scan-Identify-Clear-Flip) PP250 garbage collection algorithm but with a bidirectional G-bit integrated into both mLoad and mSave. Version-based GT invalidation prevents use-after-free. GC is implemented as a safe Turing abstraction — an atomic Turing machine behind a Church-callable entry.

COMPLETE CLAIMS

Claims are grouped by filing. All claims in each group are numbered sequentially across the full document.

PART I — BASE PATENT

Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination Filed February 2026

Claim 1 — GT Type Field with NULL and Abstract Types

A processor architecture comprising a capability register file wherein each register holds a Golden Token (GT) having a Type field of at least two bits that architecturally classifies the token content as one of: a local reference (Inform, Type = 00), a remote reference (Outform, Type = 01), a null/empty/invalid capability (NULL, Type = 10), or an unforgeable constant value (Abstract, Type = 11, e.g., mathematical or physical constants such as pi); wherein the Type field is checked by hardware at each instruction to determine the execution path; wherein a NULL-typed token causes an immediate hardware fault on any operation, providing an unambiguous representation for empty, uninitialized, or revoked capability registers; and wherein an Abstract-typed token encodes an immutable value that requires no namespace dereference, enabling hardware-protected constants that cannot be forged, modified, or confused with capabilities.

Claim 2 — NULL Type for Initialization, Revocation, and Garbage Collection

The architecture of Claim 1, wherein the NULL type serves three architectural roles:

1. initialization, wherein freshly created threads have all non-essential capability registers set to NULL, providing a clean and unambiguous initial state;
 2. revocation, wherein revoking a capability sets the corresponding register to NULL, causing any subsequent use to FAULT immediately rather than encountering ambiguous state;
 3. garbage collection, wherein the GC scanner distinguishes NULL-typed registers (no namespace entry to scan) from valid Inform or Outform GTs (namespace entries to mark as reachable), eliminating the ambiguity between an empty register and a valid capability with index zero.
-

Claim 3 — LAMBDA Instruction

The architecture of Claim 1, further comprising a LAMBDA instruction having two operands: a capability register (CRn) holding a Golden Token with

Execute (X) permission referencing executable code, and a data register (x) holding an argument value; wherein said LAMBDA instruction:

1. verifies that CRn holds a token of Inform type (00) with Execute (X) permission;
2. saves the return address (PC+4) to a machine status register;
3. sets a LAMBDA-active flag in machine status;
4. branches to the code referenced by CRn for execution within the same protection domain, without changing the current capability list, without allocating a stack frame, and without performing namespace revalidation;
5. allows the code body to operate on argument and result values through data registers without writing to capability registers;

thereby implementing Church's lambda calculus function application as a lightweight in-scope code invocation at reduced overhead compared to a domain-crossing CALL instruction, achieving macro-like code reuse without code duplication.

Claim 4 — LAMBDA vs. CALL Distinction

The architecture of Claim 3, wherein the LAMBDA instruction uses Execute (X) permission and operates within the current protection domain without stack frame allocation or capability list switching; and wherein a separate CALL instruction uses Enter (E) permission and crosses protection domain boundaries with stack frame allocation, capability list switching, and full mLoad validation; said distinction corresponding to Church's lambda calculus application within a domain versus service invocation across domains.

Claim 5 — Machine-Status Fast Path for LAMBDA Return

The architecture of Claim 3, wherein the LAMBDA instruction stores the return address and LAMBDA-active flag in dedicated machine status registers rather than on the capability stack; and wherein the RETURN instruction, upon detecting the LAMBDA-active flag in machine status, restores the program counter from the machine status register and clears the LAMBDA-active flag, completing the return with zero stack access; said machine-status fast path providing the common-case execution path for LAMBDA invocations where no CALL or CHANGE instruction intervenes during the LAMBDA body.

Claim 6 — Self-Describing Stack Frames with 1-Bit Tag

The architecture of Claims 3 and 4, wherein every frame on the capability stack carries a 1-bit tag identifying the frame as either a CALL frame (tag = 0) or a LAMBDA frame (tag = 1); and wherein the RETURN instruction, when popping a frame from the stack, inspects the tag to determine the restoration path — performing full domain restoration for CALL frames, and performing simple program counter restoration for LAMBDA frames; said tag making the thread's execution history self-describing.

Claim 7 — Non-Nestable LAMBDA with CALL-Mediated Nesting

The architecture of Claims 3 and 4, wherein a second LAMBDA instruction executed while the LAMBDA-active flag is set in machine status causes a hardware fault, preventing uncontrolled nesting; and wherein a CALL instruction executed during a LAMBDA body saves the LAMBDA machine status as part of the CALL stack frame and clears the LAMBDA-active flag in machine status; thereby permitting a nested LAMBDA instruction within the called procedure, with the outer LAMBDA state preserved on the stack and restored when the CALL returns.

Claim 8 — Clean CR/DR Separation

The architecture of Claim 1, wherein capability registers hold exclusively Golden Tokens and data registers hold exclusively numeric values; wherein no instruction transfers raw numeric values into capability registers or extracts raw bit patterns from capability registers into data registers; and wherein the LAMBDA instruction bridges the two domains by using a capability (GT with X permission in a CR) to execute code that operates on values (arguments and results in data registers), without writing to capability registers during execution.

Claim 9 — Dual-Gate Trusted Security Base (mLoad + mSave)

The architecture of Claim 1, comprising a Trusted Security Base consisting of exactly two hardware gates:

1. an mLoad read gate that validates every read-side capability operation through a sequential pipeline of permission check, bounds check, version match, MAC/seal validation, G-bit reset, capability register write, and thread table shadow update; wherein mLoad is the sole path for all

capability register writes involving namespace dereferencing; and wherein mLoad enforces a permission gate table mapping R→DREAD, W→DWRITE, X→LAMBDA, L→LOAD, S→SAVE, E→CALL;

2. an mSave write gate that validates every write of a Golden Token to a C-List through a sequential pipeline of source GT version match, source GT seal validation, target C-List bounds check, B-bit check (B=1 required for bindability), F-bit detection on target slot, seal recomputation, G-bit reset, and commit; wherein mSave is the sole path for all capability writes to C-Lists;

wherein the dual-gate TSB provides symmetric validation on both read and write paths, and the total TSB comprises fewer than 400 lines of synthesizable hardware description language; and wherein any failure at any step in either gate routes to a single hardware FAULT handler with no partial state, no silent failure, and no fallback path.

Claim 10 — B (Bind) Bit for Capability Propagation Control

The architecture of Claims 1 and 9, further comprising a B (Bind) bit stored as namespace entry metadata (word1, bit 31) that controls capability propagation; wherein:

1. B defaults to 0 on all namespace entries, meaning capabilities are non-bindable by default and cannot be saved to another C-List;
2. the mSave write gate checks B=1 on the source GT before committing a write to a C-List, and FAULTs if B=0;
3. CALL auto-clears B on all preserved capability registers passed to the callee, preventing the callee from propagating the caller's capabilities to other domains;
4. Allow Bind is the explicit special case, requiring TPERM with B modifier before CALL to set B=1 on a specific capability;

thereby providing hardware-enforced control over capability distribution across protection domains, with the secure default being non-propagation.

Claim 11 — Network-Transparent RPC via Namespace-Stored Tunnel Key and F-bit Routing

The architecture of Claims 1 and 9, wherein a cryptographic key for a point-to-point RPC tunnel between two Meta Machines is stored in a standard namespace entry, accessed via CAPLOAD with Read (R) permission through the mLoad validation path; wherein the F (Far/Foreign) flag on namespace entries marks remote resources; wherein on the read path, mLoad detects $F=1$ and routes the access through HTTPS fetch (for R permission) or RPC tunnel (for E permission); wherein on the write path, mSave detects $F=1$ on the target slot and routes the write through HTTPS flush (for W permission) or RPC tunnel; and wherein garbage collection of the tunnel key's namespace entry instantly revokes the tunnel by invalidating all copies of the GT that references the key entry.

Claim 12 — Deterministic Garbage Collection with Bidirectional G-bit

The architecture of Claims 1 and 9, wherein deterministic garbage collection comprises a four-phase cycle (Scan-Identify-Clear-Flip) with a bidirectional G-bit mechanism; wherein:

1. the G-bit is reset on every namespace access through both the mLoad read gate and the mSave write gate, ensuring that reachability determines liveness regardless of whether the access is a read or write operation;
2. the Scan phase walks the reachability tree from all live roots, clearing $G=0$ on reachable entries;
3. entries still marked $G=1$ after scanning are unreachable and have their version bumped, instantly invalidating all outstanding Golden Tokens that reference the old version;
4. garbage collection is implemented as a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry, entered via CALL and exited via RETURN;

thereby preventing use-after-free vulnerabilities through deterministic version-based invalidation with zero runtime overhead on the fast path.

Claim 13 — M Permission as Transient Microcode Elevation

The architecture of Claim 1, further comprising a seventh permission bit M (Meta/Microcode) that exists only transiently on capability registers during microcode execution and is never stored in Golden Tokens; wherein the microcode elevates M on a CR to perform privileged actions including

namespace reads, namespace writes, thread state updates, and garbage collection scanning; wherein M is cleared when the microcode operation completes; and wherein no user instruction can set, test, or observe M; thereby providing privileged microcode access without requiring a privileged hardware mode, supervisor state, or kernel trap mechanism.

Claim 14 — Atomic Abstraction Architecture with Zero-OS Security

The architecture of Claims 1, 3, and 9, wherein the processor operates without a central operating system, without virtual memory, without privileged hardware modes, and without a superuser identity; wherein all system services are atomic abstractions accessed exclusively through Golden Tokens; wherein the dual-gate TSB provides the sole trusted path for all capability operations; and wherein the architecture achieves seven security zeros: zero OS required, zero virtual memory, zero privilege escalation, zero superuser, zero unauthorised code execution, zero unauthorised data access, and zero containment escape.

Claim 15 — Five-Phase Hardware Boot with NULL Initialization

The architecture of Claims 1 and 2, wherein the processor boots through a five-phase hardware sequence: (0) IDLE; (1) FAULT_RST, clearing all CRs to NULL type, all DRs to zero; (2) LOAD_NS, loading the namespace GT into CR15 from a hardwired bootstrap source; (3) INIT_THRD, initialising CR8 and CR5; (4) LOAD_NUC, loading CR14 and CR6; (5) COMPLETE, beginning instruction fetch; wherein Phase 1 sets all capability registers to NULL type, and Phases 2-4 load valid GTs through mLoad.

Claim 16 — Mint as Domain-Pure Namespace Method

The architecture of Claim 1, wherein the operation to create new Golden Tokens (Mint) is a method of the Namespace abstraction accessed through a chain of abstraction nesting hidden behind capability boundaries; wherein the Mint operation enforces domain purity by requiring that access rights be either Turing-domain (any combination of R, W, X) or Church-domain (any combination of L, S, E), never both; and wherein the B-bit on newly minted capabilities defaults to 0 (non-bindable).

Claim 17 — Pure Church Lambda Processor

A processor architecture comprising:

1. a software-accessible instruction set consisting exclusively of lambda calculus reduction operations: LOAD (L permission), SAVE (S permission), CALL (E permission), RETURN, LAMBDA (X permission), and TPERM (permission verification);
2. wherein no arithmetic instruction, no logic instruction, no comparison instruction, no branch instruction, no direct memory addressing instruction, and no register transfer instruction is available to software;
3. wherein all arithmetic computation is performed through Church-encoded lambda calculus reductions: Church numerals for natural numbers, Church booleans for conditional logic, Church pairs for structured data, and Y-combinator for recursive computation;
4. wherein every instruction operates exclusively through Golden Tokens with hardware-verified permissions, validated by the dual-gate TSB of Claim 9, and every failure routes to a single FAULT handler.

Claim 18 — Security Enforcement Through Architectural Instruction Exclusion

The processor of Claim 17, wherein the architectural exclusion of Turing-domain instructions from the software instruction set eliminates, by construction rather than by mitigation:

1. buffer overflow attacks, because no instruction can write to a computed arbitrary address;
 2. return-oriented programming attacks, because no branch instruction exists to chain code gadgets;
 3. code injection attacks, because no instruction can write executable code (domain purity prevents S and X permissions on the same GT) and no instruction can branch to an arbitrary address;
 4. privilege escalation, because no operating system, privilege rings, or superuser identity exists and no instruction can forge or modify a Golden Token;
 5. use-after-free exploits, because TPERM verifies capability validity before every operation and revoked capabilities cause immediate FAULT.
-

Claim 19 — Hardware I/O Mediator for Pure Lambda Processor

The processor of Claim 17, further comprising a hardware I/O mediator module that:

1. is the sole hardware interface between pure lambda software and physical devices;
 2. intercepts SAVE instructions targeting Golden Tokens with the F (Far) flag set, wherein the namespace entry identifies a physical device class;
 3. translates Church-encoded output values into physical bus transactions;
 4. intercepts LOAD instructions on device-class Golden Tokens and returns hardware status as Church-encoded values;
 5. enforces Golden Token permissions on all device access through the mLoad validation path;
 6. is architecturally equivalent to a single trusted gate for the physical world and cannot be bypassed by any software instruction.
-

Claim 20 — Church Numeral Method-Selector Dispatch

The processor of Claim 17, further comprising a method-selector dispatch mechanism wherein:

1. a method selector value is converted to a Church numeral by applying the Church successor function DR1 times to Church zero using the LAMBDA instruction;
2. the resulting Church numeral indexes into the abstraction's C-List to obtain the corresponding method GT;
3. the method GT is verified via TPERM and applied via LAMBDA;
4. no branch instruction, jump table, computed goto, or conditional logic instruction is used;

thereby implementing polymorphic method dispatch entirely through lambda calculus.

Claim 21 — Church-Encoded Arithmetic via Capability Tokens

The processor of Claim 17, wherein arithmetic operations including at least addition, subtraction, multiplication, division, modular arithmetic, exponentiation, logarithm, and square root are performed exclusively through:

1. Church numeral encoding;
2. capability-mediated function application, wherein each arithmetic primitive is a Golden Token in the Lambda abstraction's C-List, loaded via LOAD with L permission and applied via LAMBDA with X permission;
3. recursive operations using the Y-combinator Golden Token with Church LEQ for termination and Church SUCC/PRED for iteration;
4. composite operations expressed by composing Church primitives through sequential LAMBDA applications, without any Turing-domain arithmetic instruction.

Claim 22 — Three-Block Pure Church Processor Architecture

The processor of Claim 17, comprising exactly three hardware functional blocks:

1. a Lambda Reducer that executes the six Church-domain instructions and contains no arithmetic logic unit, no barrel shifter, no condition flag register, and no branch prediction unit;
2. a Capability Validator implementing the dual-gate TSB of Claim 9;
3. an I/O Mediator as in Claim 19;

wherein the total processor comprises fewer functional units than a conventional processor, resulting in smaller silicon area, lower power consumption, and a design amenable to formal verification.

Claim 23 — Interactive Pure Church Programming Model

A method of programming the pure Church lambda processor of Claim 17, maintaining the security properties of Claim 18, comprising:

1. an interactive execution environment (REPL) wherein each expression is parsed, translated to capability-secured Church-domain operations, and executed through the complete capability-checked pipeline (LOAD → TPERM → CALL → LOAD → TPERM → LAMBDA → RETURN);

2. named variable bindings following the step-by-step named-result programming style first described by Ada Lovelace in Note G (1843);
 3. program file execution with persisting variable scope enabling multi-step computations expressed entirely in Church-domain instructions;
 4. fail-safe error handling, wherein undefined variable references produce explicit errors and Turing-domain instructions produce immediate FAULT;
 5. wherein the programming model demonstrates general-purpose interactive programming with the security properties of Claim 18 for every computation.
-

Claim 24 — Safe Turing Abstractions with Church Interface

The architecture of Claims 1 and 9, wherein Turing-domain implementations are encapsulated inside Church-callable namespace entries; wherein:

1. the abstraction is entered only via CALL with valid E-permission GT or LAMBDA with valid X-permission GT;
2. internal Turing instructions execute atomically within the abstraction, invisible to the caller;
3. the abstraction is exited only via RETURN;
4. the caller sees only a Church interface and cannot observe or access the internal Turing implementation;

thereby providing Church-domain security properties for the interface while enabling Turing-domain computational efficiency for the implementation.

Claim 25 — DATA Objects Bridging Church and Turing Domains

The architecture of Claims 1 and 9, further comprising DATA objects that are namespace entries accessed via Turing-domain DREAD and DWRITE instructions; wherein:

1. DREAD requires R (Read) permission on the Golden Token, validated through mLoad;
2. DWRITE requires W (Write) permission on the Golden Token, validated through mLoad;

3. bounds validation ensures all reads and writes fall within the namespace entry's Location..Limit range;
4. the Golden Token providing access is a Church-domain capability while the data operations are Turing-domain instructions;

thereby bridging Church and Turing domains through capability-mediated data access with hardware-enforced permission and bounds checking.

Claim 26 — Unified Address Space Under Capability Protection

The architecture of Claims 1 and 9, wherein memory, attached devices, and machine registers occupy segments of a single flat address space; wherein:

1. memory addresses occupy MSB range 0x00-0xFD;
2. attached device registers occupy MSB 0xFE;
3. machine register bank occupies MSB 0xFF;
4. every segment is protected by the same Golden Token validation through the mLoad read gate;
5. without a valid Golden Token with appropriate permissions, any address range is unreachable;

thereby eliminating the need for separate I/O instructions, I/O privilege levels, or memory-mapped I/O permission mechanisms.

Claim 27 — Three Dispatch Styles for Abstraction Method Resolution

The architecture of Claims 3 and 4, wherein an abstraction's nucleus code may resolve method calls using any of three dispatch styles, selected by the abstraction's creator and invisible to the caller:

1. a symbolic resolver style providing maximum isolation;
2. a LAMBDA fast-path style using the LAMBDA instruction with X permission and zero stack access;
3. a traditional compiled binary style;

wherein all three styles present the same interface to the caller, and the caller cannot determine which dispatch style is used; and wherein Style (b) is made possible exclusively by the LAMBDA instruction of Claim 3.

Claim 28 — LAMBDA as Macro-Like Code Reuse

The architecture of Claim 3, wherein the LAMBDA instruction enables a code body stored once in memory to be invoked from multiple call sites with near-zero overhead per invocation and without code duplication; wherein each invocation uses the machine-status fast path (zero stack access) when no CALL or CHANGE intervenes; and wherein the code body receives arguments and returns results through data registers, operating as a reusable function that achieves the performance of an inline macro without replicating the code base.

PART II — ADDENDUM A: UNIVERSAL COMPUTATION TARGET

Language-Independent Capability-Secured Instruction Set Architecture with Multi-Language Compiler, Resident Object Model, and Upload-Driven Abstraction Lifecycle Filed March 2026

Claim 29 — Universal Computation Target Instruction Set

A processor instruction set architecture comprising:

1. a fixed set of instructions divided into two domains — a Church domain for capability operations (LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, XLOADLAMBDA) and a Turing domain for data operations (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR);
2. wherein all instructions share a uniform encoding format comprising an opcode field, a condition code field, a destination register field, a source register field, and an immediate value field;
3. wherein the instruction set constitutes a universal computation target, demonstrated by the successful compilation of source programs from at least two fundamentally different programming paradigms — imperative and functional — to the same instruction set, producing semantically equivalent machine code for equivalent operations;
4. wherein the hardware security model — comprising capability token validation, domain purity enforcement, bounds checking, and permission verification through a trusted gate pipeline — applies identically to all compiled code regardless of source language;

5. wherein the compiler is architecturally outside the trusted computing base, such that no compiler bug in any source language can produce a security violation, because the hardware enforces capability invariants independent of the code generated.

Claim 30 — Multi-Language Capability Compiler with Resident Object Model

A compilation system for the processor of Claim 29, comprising:

1. a multi-front-end compiler architecture wherein each front-end parses a different source language and all front-ends share a common back-end that generates instructions from the fixed instruction set of Claim 29;
2. a language detection mechanism that automatically identifies the source language from syntactic markers before parsing begins;
3. a Resident Object Model that maps source-language references to external abstractions (capability names) to hardware c-list offsets, wherein the mapping is derived directly from the upload's capability declaration — the same source of truth used by the namespace authority to populate the physical c-list at installation time;
4. a fixed calling convention enforced across all source languages, wherein a first set of data registers is designated for argument passing and return values, a second set for callee-saved local variables, and a third set for caller-saved temporaries;
5. wherein the Resident Object Model ensures that a capability reference in source code compiles to the same hardware instruction sequence — a LOAD from the correct c-list offset followed by a CALL — regardless of source language;
6. wherein the compiler output is a self-describing upload object containing the abstraction name, capability declarations, and compiled method code, suitable for processing by the namespace authority of Claim 33.

Claim 31 — Single-Lump Abstraction Model with Hardware CALL Split

A method of managing abstractions in the processor of Claim 29, comprising:

1. allocating a single contiguous memory region (lump) for each abstraction, wherein the lump contains both compiled code and a capability list (c-list);
2. describing the lump with a single namespace entry, wherein the namespace entry's word1 field contains a clistCount value indicating the number of c-list entries;
3. upon execution of a CALL instruction targeting the abstraction:
computing a split point as $\text{clistStart} = (\text{limit} + 1) - \text{clistCount}$;
creating a first context register (CR14) pointing to the code region with permissions hardcoded to execute-only (X); creating a second context register (CR6) pointing to the c-list region with permissions hardcoded to load-only (L); setting the program counter to zero;
4. wherein the code region permissions (X-only) and c-list region permissions (L-only) are architectural constants enforced by the CALL instruction, not derived from the Golden Token or namespace entry permissions;
5. wherein this hardcoded domain split prevents code from reading capabilities as data and prevents c-list entries from being executed as code, maintaining domain purity across the code/capability boundary.

Claim 32 — Capability-Type Taxonomy with Architectural Semantics

The processor of Claim 29, wherein each Golden Token contains a two-bit type field with the following architectural semantics:

1. NULL (00): zero value, no capability; any dereference produces an immediate FAULT;
2. Inform (01): the GT points to a memory-backed object described by a namespace entry; the clistCount field distinguishes abstractions ($\text{clistCount} > 0$) from plain data objects ($\text{clistCount} = 0$);
3. Outform (10): the GT points to a resource in a remote namespace; the Far (F) bit is set automatically; used for cross-namespace and network-transparent access;

4. Abstract (11): the GT IS the value; no memory pointer or namespace entry is required; used for mathematical constants, immutable credentials, and escale variables;

wherein the type field is validated by the hardware trusted gate pipeline on every access, and type-specific behaviour is performed by the hardware without software intervention.

Claim 33 — Sole Namespace Authority with Upload-Driven Lifecycle

A method of managing the namespace table in the processor of Claim 29, comprising:

1. designating a single abstraction (Navana) as the sole authority permitted to write namespace table entries after the boot sequence;
2. during boot, performing exactly one privileged write (mElevation) to install Navana's own namespace entry, and thereafter permanently relinquishing privileged write access;
3. routing all subsequent namespace table writes through Navana.Add, which finds a free namespace slot, writes the three-word namespace entry, and returns the namespace index and version;
4. processing abstraction creation through Navana.Abstraction.Add, which validates the upload object, allocates a power-of-2 memory lump, writes compiled code to the code region, populates the c-list with delegated GTs, creates the namespace entry, and forges an Inform E-GT returned to the creator;
5. wherein other abstractions (including Mint) delegate namespace entry creation to Navana, ensuring a single point of namespace authority for all abstractions regardless of source language or creation method.

Claim 34 — Compiler-Security Architectural Separation

The compilation system of Claim 30, wherein:

1. the compiler is architecturally outside the trusted computing base of the processor;
2. a compiler bug can produce functionally incorrect code but cannot produce a security violation;

3. security invariants enforced by the hardware independent of compiler output include: Golden Token validation on every memory access; domain purity (Church and Turing permissions cannot coexist on a single GT); bounds checking; CALL split hardcoding (CR14=X-only, CR6=L-only regardless of compiled code); c-list authority (code can only LOAD GTs present in its c-list; no GT can be forged by any instruction sequence);
 4. wherein this separation holds for all source languages, such that adding a new language front-end cannot weaken the security model — the hardware provides a language-independent security floor.
-

Claim 35 — Cross-Paradigm Capability Interoperability

The compilation system of Claim 30, wherein:

1. abstractions compiled from different source languages can invoke each other through the standard CALL mechanism, using the same E-GT protocol, register convention, and c-list wiring;
 2. a first abstraction compiled from an imperative language and a second compiled from a functional language can hold E-GTs to each other in their respective c-lists;
 3. the CALL instruction, the mLoad validation pipeline, and the lump split mechanism operate identically regardless of the source languages of the caller and callee;
 4. no adaptation layer, foreign function interface, or runtime type conversion is required for cross-paradigm invocation — the hardware protocol is the sole interface.
-

Claim 36 — Scale-Free Security Architecture

The processor of Claim 29, wherein:

1. the abstraction model (GT + namespace entry + lump + CALL split) applies identically at every organisational scale — individual user, family, school, enterprise, and inter-organisational;
2. namespace isolation is achieved by each entity having its own namespace table, memory region, and set of Golden Tokens;

3. access across namespace boundaries requires a valid Outform GT (type=10) with the Far bit set, processed through the same mLoad validation pipeline;
4. capability delegation across scales uses the same upload protocol and Navana validation path;
5. revocation at any scale is achieved by incrementing the version number in the namespace entry, instantly invalidating all outstanding GT copies — with no revocation-list propagation delay;
6. no admin mode, superuser privilege, root access, or enterprise tier exists — the same 20 instructions, the same mLoad pipeline, and the same CALL split apply to every user at every scale.

PART III — ADDENDUM B: ABSTRACT GT I/O AND NETWORK ADDRESSING

Church-Turing Meta-Machine: Abstract Golden Token I/O and Network Addressing Architecture Enabling Hardware-Enforced Guaranteed Crime-Free Business Services Filed March 2026

Claim 37 — Abstract Golden Token with Hardware-Routed Sentinel Address

A capability register architecture wherein a Golden Token of type Abstract (gt_type = 11₂) contains a word1_location field that is a 32-bit sentinel address outside the real RAM range, and wherein hardware matches this address directly against an Abstract Address Space table rather than performing a namespace lookup, such that the address alone constitutes the token's identity and unforgeability, without any namespace slot allocation, CRC validation, or lump header dereference.

Claim 38 — Home Base Tunnel as Single Network Gateway

An I/O virtualisation mechanism wherein a single Abstract GT (Home Base tunnel at address 0xFF000000) is the sole outbound network interface, with all network access from all abstractions routed through this single hardware-managed tunnel, and wherein the Home Base GT can be provisioned with optional programmer-defined backup IDE addresses (Word 2 and Word 3) that are tried in sequence if the primary address is unreachable, with all three addresses set atomically at boot and immutable thereafter.

Claim 39 — Local Peripheral Autonomy Without IDE Assistance

A hardware bootstrap mechanism wherein attached peripherals (UART, GPIO, Timer, Display, Storage) are identified and Abstract GTs are provisioned entirely during local hardware boot, before any user code runs and before any IDE connection is established, such that the CTMM maintains full security control over local I/O without any remote authority, enabling air-gapped, offline operation where the CTMM is the sole authority for its own hardware security policy.

Claim 40 — MTBF Qualification as Hardware-Enforced Downloadability Gate

A hardware-tracked reliability mechanism wherein every Secure Abstraction carries `invocation_count` and `failure_count` counters, an MTBF score is computed as `invocation_count / (failure_count + 1)`, and the S (Save) permission on the abstraction's GT is hardware-locked based on the MTBF tier (Isolated, User-regulated, Namespace-regulated), such that unreliable abstractions cannot propagate beyond their provisioning context, and the IDE remotely updates tier thresholds via signed policy payloads delivered through the Home Base tunnel.

Claim 41 — Structural Capability Scoping for Crime-Free Services

A service provisioning architecture wherein a CTMM provisioned for a specific profession, language, nationality, or age group receives only Abstract GTs for services within that scope, such that any code running on the CTMM cannot access services outside the scope because the capability token does not exist, providing a structural guarantee (not a policy-based filter) that access is impossible regardless of exploits, VPN tunnels, or code injection — because the hardware rejects any operation without a valid GT.

Claim 42 — Two-Tier Backup IDE Fallback with Atomic Provisioning

An extension of Claim 38 wherein the Home Base GT includes `word2_backup1` and `word3_backup2` fields, each containing a 32-bit Abstract Address within the tunnel range, and hardware implements sequential failover: try primary → try Word 2 if reachable → try Word 3 if reachable → TRAP: `TUNNEL_UNAVAILABLE`, with loop detection ensuring no two addresses are

identical and no recently-tried address is re-attempted within the same connection attempt.

Claim 43 — MTBF Telemetry and Distributed Reputation

An extension of Claim 40 wherein MTBF counters are sent to the IDE via Home Base W permission, signed with HMAC-SHA256, and the IDE maintains a permanent reputation record per abstraction per CTMM, such that an abstraction's reliability history is shared across the fleet: low MTBF on one CTMM lowers the threshold for all CTMMs through policy update, and high MTBF qualifies abstractions for namespace-tier distribution.

Claim 44 — Secure Individuality via Unique GT Sets

A trust isolation mechanism wherein each abstraction's identity is defined as the unique set of GTs provisioned in its c-list, such that no two abstractions share ambient authority, an attacker who compromises one abstraction gains only its GTs and cannot escalate to another abstraction's resources without holding that abstraction's GTs, and the privileged superuser attack window is eliminated entirely — there is no privileged layer that holds authority on behalf of others.

Claim 45 — Capability-Gated Denial-of-Service Prevention

A denial-of-service prevention mechanism wherein access to any resource is controlled exclusively through unforgeable capability tokens, such that an attacker without the capability token for a resource cannot even request operations on that resource, and thus cannot launch a DoS attack on that resource, with the constraint being structural and enforced at the hardware level rather than through rate-limiting or filtering.

Claim 46 — Per-Abstraction Resource Isolation

A resource isolation mechanism wherein each Secure Abstraction has its own c-list with its own GT set, and resources (I/O, network, memory allocators, queues) are not shared by default, such that an attacking abstraction that exhausts its own resources affects only itself and does not cascade to other abstractions, enabling DoS containment at the abstraction boundary.

Claim 47 — Hardware-Enforced Retry Limits on Critical Paths

A rate-limiting mechanism wherein critical paths (mLoad for namespace access, SWITCH for privilege gate, GC for garbage collection) enforce a maximum of three contention retries before raising a TRAP that returns control to the IRQ handler, and wherein the hardware counter resets only on successful completion or explicit FAULT, such that an attacker cannot hammer critical paths without explicit wait periods, preventing saturation DoS attacks on the Trusted Security Base.

Claim 48 — MTBF Qualification as DoS Containment

An extension of Claim 40 wherein an abstraction that fails frequently is automatically downgraded to Isolated tier and cannot be propagated to other CTMMs, such that DoS-prone or malicious abstractions cannot spread across the fleet through normal capability propagation, and fleet-wide policy updates further tighten MTBF thresholds to quarantine unreliable abstractions.

Claim 49 — Per-Abstraction Bandwidth Quotas on Tunnel Access

An extension of Claim 38 wherein the Home Base tunnel implements per-abstraction outbound bandwidth quotas (measured in bytes per second or operations per second), and wherein abstractions that exceed their quota are backpressured rather than dropped, such that one attacking abstraction cannot saturate the shared tunnel and starve other abstractions of network access.

Claim 50 — Local Peripheral Access Rate Limiting

An extension of Claim 39 wherein access to local peripherals (UART, GPIO, Timer, Display) is rate-limited per abstraction through hardware quotas or token-bucket mechanisms, such that an abstraction cannot monopolise peripheral bandwidth and deny-of-service other abstractions' local I/O operations.

PART IV — ADDENDUM C: LAMBDA RECURSION AND SELF-INVOCATION

Idempotent LAMBDA CR6 Self-Invocation for O(1) Recursion in a Capability-Based Processor, with Natural Language Compilation and Pet-Name Mathematical Constants Filed April 2026

Claim 51 — Self-Invocation via Capability List Register

A recursion mechanism in a capability-based processor comprising:

1. a CALL instruction that, upon entering an abstraction, creates a code region capability register (CR14) and a capability list register (CR6) by splitting the abstraction's memory lump;
2. wherein the capability list register (CR6) holds an Inform Golden Token pointing to the abstraction's own entry point, providing a self-reference as a free consequence of the CALL lump split — no additional GT allocation, namespace lookup, or c-list modification is required;
3. wherein a LAMBDA instruction targeting CR6 re-enters the same abstraction with X permission verification only, no namespace gate swap, and no stack frame push — implementing optimal recursive self-invocation with $O(1)$ entry, $O(1)$ context switch, and $O(1)$ exit;
4. wherein a CALL instruction targeting CR6 also re-enters the same abstraction but with full namespace gate re-validation, a new 2-word stack frame, and $O(N)$ cost — providing no additional security over LAMBDA CR6 because the re-validation re-checks the same GT every time;
5. wherein the architectural analysis demonstrates that LAMBDA CR6 dominates CALL CR6 for self-invocation on every axis: identical security (same method, same domain, same capabilities), but $O(1)$ cost versus $O(N)$.

Claim 52 — Idempotent LAMBDA Re-Entry for $O(1)$ Recursion

A hardware mechanism for recursive self-invocation in a capability-based processor, comprising:

1. a LAMBDA instruction that, when targeting the capability list register (CR6) while the `lambda_active` flag is already set, is permitted to re-execute without generating a FAULT — because the return address written to `lambda_pc` is invariant (always PC+4 of the same LAMBDA CR6 instruction), making the re-entry idempotent;
2. wherein the method's own recursion argument drives the recursion — the hardware does not independently track recursion depth;

3. wherein the base-case RETURN instruction, upon detecting `lambda_active = 1`, restores PC from `lambda_pc` and clears `lambda_active` in a single cycle with no stack access;
 4. wherein a second RETURN instruction, finding `lambda_active = 0`, performs the real RETURN by popping the CALL frame from the initial method entry;
 5. wherein exactly two RETURN instructions execute regardless of recursion depth — achieving $O(1)$ exit for arbitrarily deep recursion with no hardware counter, no stack unwinding, and no iterative frame popping;
 6. wherein the FAULT is preserved for LAMBDA to a different target while `lambda_active` is set — the idempotent exception applies exclusively to CR6 self-invocation;
 7. thereby achieving $O(1)$ recursion entry, $O(1)$ context switch, and $O(1)$ exit — using only the existing `lambda_active` flag and `lambda_pc` register with no additional hardware.
-

Claim 53 — Two-RETURN Exit Path

The processor of Claim 52, wherein:

1. the code structure of a LAMBDA CR6 recursive method places the LAMBDA CR6 instruction as the final executable statement before the method's trailing RETURN instruction;
 2. the base-case RETURN (#1) clears `lambda_active` and jumps to the instruction after LAMBDA CR6, which is the method's trailing RETURN;
 3. the trailing RETURN (#2) finds `lambda_active = 0` and pops the CALL frame from the initial method entry;
 4. thereby, the two-RETURN exit path is a structural consequence of the method layout, regardless of whether recursion depth was 1, 100, or 1,000,000.
-

Claim 54 — Three Architectural Loop Styles Demonstrating LAMBDA CR6 Dominance

A processor architecture providing three distinct loop mechanisms within a single instruction set, wherein analysis demonstrates that LAMBDA CR6 is the optimal recursion primitive:

1. a compare-and-branch loop (While) using MCMP and BRANCH instructions, which is familiar to imperative programming but requires branch prediction, is susceptible to pipeline stalls on misprediction, and enables speculative execution vulnerabilities (Spectre, Meltdown);
2. a recursive repeat (CALL CR6) which invokes the current method through the full CALL path with namespace gate re-validation on every iteration — wherein the re-validation is redundant because CR6 always points to the same method, adding $O(N)$ stack and context-switch cost with no security benefit over LAMBDA CR6;
3. a lambda recursion (LAMBDA CR6) which invokes the current method through the idempotent LAMBDA re-entry path of Claim 52, with X permission check only, no branch prediction, no speculative execution, no namespace swap, no stack frames, and $O(1)$ entry, context switch, and exit — requiring no hardware beyond the existing `lambda_active` flag and `lambda_pc` register;
4. wherein all three mechanisms compile from the same source language to the same instruction set and produce the same computational result, but LAMBDA CR6 dominates: same security as CALL CR6, $O(1)$ cost instead of $O(N)$, and zero additional hardware;
5. wherein LAMBDA CR6 requires no additional hardware — the existing `lambda_active` flag and `lambda_pc` register, already present for single-level LAMBDA in the base patent, are the complete recursion mechanism.

Claim 55 — Natural Language Compilation to Capability-Secured Instructions

A compilation method for a capability-secured processor, comprising:

1. an English-language front-end that parses structured English sentences including method declarations (“Add a method called X that takes n”), variable assignments (“Set total to total plus n”), conditional blocks (“While n is greater than 0 ... End while”, “If n is equal to 0 ... End if”), and recursive self-invocation (“Repeat with n, total”, “Apply lambda with n, total”);

2. wherein the English front-end compiles to the same capability-secured instruction set as the JavaScript (imperative) and Haskell (functional) front-ends;
 3. wherein the English front-end exposes all three loop styles of Claim 54 through natural language syntax: “While ... End while” for compare-and-branch, “Repeat with” for secure recursive repeat, and “Apply lambda with” for lightweight lambda recursion;
 4. wherein the compiled output is indistinguishable from the output of the JavaScript or Haskell front-ends for equivalent computations — the hardware cannot determine the source language;
 5. thereby extending the universal computation target property to three programming paradigms: imperative, functional, and natural language.
-

Claim 56 — Invariant Return Address as Idempotent Re-Entry Precondition

The processor of Claim 52, wherein the correctness of idempotent re-entry depends on a structural invariant:

1. every LAMBDA CR6 instruction within a given method body branches to the same target (the method’s own code entry point, as established by CR6);
 2. every LAMBDA CR6 instruction writes the same return address to `lambda_pc` (the instruction immediately following LAMBDA CR6);
 3. the return address is invariant across all recursion levels — it does not change with recursion depth — making each re-entry write idempotent;
 4. therefore re-entry does not create new return context — the `lambda_pc` register holds the same value before and after the write — and no stack, counter, or additional state is needed;
 5. this invariant holds exclusively for LAMBDA CR6 self-invocation; LAMBDA to a different target writes a different return address and would corrupt the existing `lambda_pc`, which is why the non-nestable FAULT is preserved for non-CR6 targets.
-

Claim 57 — Pet-Name Mathematical Constants via Abstract GT

A method of representing mathematical constants in a capability-based processor, comprising:

1. each constant (Pi, E, Phi, Zero, One) is represented as an Abstract Golden Token (gt_type = 11₂) in the abstraction's c-list;
2. the compiler resolves the constant name to its c-list offset and emits a LOAD instruction to retrieve the Abstract GT into a capability register;
3. the constant's value is encoded in the Abstract GT's word1_location field and is architecturally immutable — no instruction can modify it;
4. the constant is unforgeable — no instruction can synthesise a GT, and the constant can only be accessed through capability-mediated LOAD with L permission on CR6;
5. thereby providing mathematical constants as first-class capability-secured values, accessed through the same GT mechanism used for I/O peripherals and network tunnels.

CLAIM DEPENDENCY MAP

Claim	Depends On	Subject
1	—	GT Type Field (NULL, Abstract)
2	1	NULL Type roles
3	1	LAMBDA Instruction
4	3	LAMBDA vs. CALL
5	3	Machine-Status Fast Path
6	3, 4	Self-Describing Stack Frames
7	3, 4	Non-Nestable LAMBDA
8	1	CR/DR Separation
9	1	Dual-Gate TSB (mLoad + mSave)
10	1, 9	B-bit Propagation Control
11	1, 9	Network-Transparent RPC
12	1, 9	Deterministic GC
13	1	M-Permission Microcode
14	1, 3, 9	Zero-OS Atomic Architecture
15	1, 2	Five-Phase Boot

Claim	Depends On	Subject
16	1	Mint Domain-Pure
17	—	Pure Church Processor
18	17	Security by Exclusion
19	17	I/O Mediator
20	17	Church Numeral Dispatch
21	17	Church Arithmetic
22	17, 9, 19	Three-Block Architecture
23	17, 18	Interactive REPL
24	1, 9	Safe Turing Abstractions
25	1, 9	DATA Objects
26	1, 9	Unified Address Space
27	3, 4	Three Dispatch Styles
28	3	LAMBDA as Macro
29	—	Universal Target ISA
30	29	Multi-Language Compiler + ROM
31	29	Single-Lump CALL Split
32	29	GT Type Taxonomy
33	29	Sole Namespace Authority
34	30	Compiler Outside TCB
35	30	Cross-Paradigm Interop
36	29	Scale-Free Security
37	—	Abstract GT Sentinel Address
38	—	Home Base Tunnel Gateway
39	—	Local Peripheral Autonomy
40	—	MTBF Downloadability Gate
41	—	Crime-Free Structural Scoping
42	38	Backup IDE Failover
43	40	MTBF Telemetry/Reputation
44	—	Secure Individuality
45	—	Capability-Gated DoS Prevention
46	—	Per-Abstraction Resource Isolation
47	—	Hardware Retry Limits

Claim	Depends On	Subject
48	40	MTBF as DoS Containment
49	38	Bandwidth Quotas
50	39	Peripheral Rate Limiting
51	—	Self-Invocation via CR6
52	—	Idempotent LAMBDA Re-Entry
53	52	Two-RETURN Exit
54	—	Three Loop Styles
55	—	Natural Language Compilation
56	52	Invariant Return Address
57	—	Mathematical Constants via Abstract GT

ABSTRACT

The Church-Turing Meta-Machine (CTMM) is a processor architecture enforcing all access control through unforgeable capability tokens (Golden Tokens, GTs), validated by a dual-gate Trusted Security Base comprising an mLoad read gate and an mSave write gate — together fewer than 400 lines of synthesisable HDL. Every GT contains a 2-bit Type field (Inform, Outform, NULL, Abstract) and six permission bits organised into mutually exclusive Turing-domain (R, W, X) and Church-domain (L, S, E) sets. A LAMBDA instruction provides lightweight in-scope code application (Church’s function application as a hardware primitive), with a machine-status fast path and zero stack access in the common case.

The architecture operates without an OS, virtual memory, privilege rings, or superuser: all system services are atomic abstractions accessed through GTs. In its Pure Church variant, all Turing-domain instructions are architecturally excluded from the software instruction set, eliminating buffer overflows, ROP attacks, code injection, and privilege escalation **by construction** — not by mitigation. Three software proofs (HP-35 calculator in 179 instructions, SlideRule in 98 instructions, interactive REPL) and synthesisable FPGA implementations demonstrate computational completeness and practical realisation.

A fixed 20-instruction universal computation target accepts compiled output from imperative (JavaScript), functional (Haskell), and natural-language (English) front-ends through a single compiler (CLOOMC++). The compiler is architecturally outside the trusted computing base; no compiler bug in any

source language can produce a security violation. A novel Abstract GT type ($gt_type = 11_2$) unifies I/O peripherals, network tunnels, and mathematical constants under the same unforgeable capability model. A single Home Base Tunnel (0xFF000000) is the sole network gateway. Structural capability scoping — the complete absence of a GT for a service, not a filter applied over it — provides verifiable, hardware-enforced crime-free service provisioning. $O(1)$ recursive self-invocation is achieved through LAMBDA CR6 idempotent re-entry using only the hardware already present for single-level LAMBDA.

Total claims: 57 Applications: 4 (Base + Addendums A, B, C) Inventor: Kenneth James Hamer-Hodges

End of Document